CS 516 Assignment 4

The goal of this assignment is to introduce the basic functionality of AI Fairness 360.

Instructions

1. Complete **all the TODO code cells** and **answer all Short Response questions (Q1–Q4)** in Markdown below each prompt.
2. Make sure your notebook runs **top-to-bottom** without errors.
3. **Export your notebook as a PDF before submitting.**
   - In Google Colab:
     - Go to `File` → `Print` → select **Save as PDF**.
     - (Or: `File` → `Download` → `Download PDF` if available.)
   - In Jupyter Notebook:
     - Go to `File` → `Download as` → `PDF via LaTeX`.
4. **Submit only the PDF** on Gradescope — `.ipynb` files are **not accepted**.
5. Ensure all code outputs (metrics, printed values, etc.) are visible in your PDF.

💡 *Tip:* If your PDF doesn't show outputs, re-run the notebook fully before exporting.

Before starting the assignment, you need to install the AI Fairness 360 (`aif360`) package(s): https://aif360.res.ibm.com/ (The AI Fairness 360 interactive demo provides a gentle introduction to the concepts and capabilities: https://aif360.res.ibm.com/data)

A comprehensive Python document with the algorithms, datasets, etc. is provided here: https://aif360.readthedocs.io/en/stable/

Here is their Github Repo: https://github.com/Trusted-AI/AIF360

Usecase: Load repay prediciton: Given an instance of a loan application, predict if the applicant will repay the loan.

Dataset: German Credit Dataset

Biases and Machine Learning

AI Fairness 360 is designed to help address the bias issues in the prediciton with *fairness metrics* and *bias mitigators*. Fairness metrics can be used to check for bias in machine learning workflows. Bias mitigators can be used to overcome bias in the workflow to produce a more fair outcome.

A bias detection and/or mitigation toolkit needs to be tailored to the particular bias of interest. More specifically, it needs to know the sensitive attirbute(s) (e.g., race, sex).

The Machine Learning Workflow

To understand how bias can enter a machine learning model, we first review the basics of how a model is created in a supervised machine learning process.

First, the process starts with a *training dataset*, which contains a sequence of instances, where each instance has two components: the features and the correct prediction for those features. Next, a machine learning algorithm is trained on this training dataset to produce a machine learning model. This generated model can be used to make a prediction when given a new instance. A second dataset with features and correct predictions, called a *test dataset*, is used to assess the accuracy of the model. Since this test dataset is the same format as the training dataset, a set of instances of features and prediction pairs, often these two datasets derive from the same initial dataset. A random partitioning algorithm is used to split the initial dataset into training and test datasets.

Bias can enter the system in any of the three steps above. The training data set may be biased in that its outcomes may be biased towards particular kinds of instances. The algorithm that creates the model may be biased in that it may generate models that are weighted towards particular features in the input. The test data set may be biased in that it has expectations on correct answers that may be biased. These three points in the machine learning process represent points for testing and mitigating bias. In AI Fairness 360 codebase, we call these points *pre-processing*, *in-processing*, and *post-processing*.

AI Fairness 360

We are now ready to utilize AI Fairness 360 (`aif360`) to detect and mitigate bias. We will use the German credit dataset, splitting it into a training and test dataset. We will look for bias in the creation of a machine learning model to predict if an applicant should be given credit based on various features from a typical credit application. The protected attribute will be "Age", with "1" (older than or equal to 25) and "0" (younger than 25) being the values for the privileged and unprivileged groups, respectively. For this first tutorial, we will check for bias in the initial training data, mitigate the bias, and recheck. More sophisticated machine learning workflows are given in the author tutorials and demo notebooks in the codebase.

Here are the steps involved:

Step 1: Write import statements

Step 2: Set bias detection options, load dataset, and split between train and test

Step 3: Compute fairness metric on original training dataset

Step 4: Mitigate bias by transforming the original dataset

Step 5: Compute fairness metric on transformed training dataset

Complete the following steps. Run the code, and show the results

Step 1 Import Statements

As with any python program, the first step will be to import the necessary packages. Below we import several components from the `aif360` package. We import the GermanDataset, metrics to check for bias, and classes related to the algorithm we will use to mitigate bias.

Add the missing lines to

1. import the germant credit dataset
2. load the binary label dataset metric and dataset metric from aif360 metrics
3. from the preprocessing methods import the Reweighing approach from aif360 algorithms

Your task:
👉 Add the missing imports to complete the setup.

- Import the **German Credit Dataset** class from `aif360.datasets`.
- Import the **BinaryLabelDatasetMetric** and **DatasetMetric** classes from `aif360.metrics`.
- Import the **Reweighing** method from `aif360.algorithms.preprocessing`.

⚠️ **Note**: If the German dataset download fails, please manually download the files from https://archive.ics.uci.edu/ml/machine-learning-databases/statlog/german/ and place them in your environment as described in the error message.

Once you're done, you should be able to run the next steps without any import errors.

In [2]:
```
# Load all necessary packages
import sys
sys.path.insert(1, "../")

import numpy as np
np.random.seed(0)

# If not already installed, uncomment and run this line:
%pip install aif360

# 👉 TODO: Import the German Credit Dataset from aif360.datasets
from aif360.datasets import GermanDataset

# 👉 TODO: Import BinaryLabelDatasetMetric (and optionally DatasetMetric) from aif
from aif360.metrics import BinaryLabelDatasetMetric, DatasetMetric

# 👉 TODO: Import the Reweighing method from aif360.algorithms.preprocessing
from aif360.algorithms.preprocessing import Reweighing
```

```
# Other necessary imports
from sklearn.model_selection import train_test_split
from aif360.explainers import MetricTextExplainer, MetricJSONExplainer
from IPython.display import Markdown, display

import matplotlib
import matplotlib.pyplot as plt
import seaborn as sns
import json
from collections import OrderedDict
```

```
Requirement already satisfied: aif360 in c:\users\sudha\appdata\local\programs\pytho
n\python310\lib\site-packages (0.6.1)
Requirement already satisfied: numpy>=1.16 in c:\users\sudha\appdata\local\programs
\python\python310\lib\site-packages (from aif360) (1.26.4)
Requirement already satisfied: scipy>=1.2.0 in c:\users\sudha\appdata\local\programs
\python\python310\lib\site-packages (from aif360) (1.15.3)
Requirement already satisfied: pandas>=0.24.0 in c:\users\sudha\appdata\local\progra
ms\python\python310\lib\site-packages (from aif360) (2.3.3)
Requirement already satisfied: scikit-learn>=1.0 in c:\users\sudha\appdata\local\pro
grams\python\python310\lib\site-packages (from aif360) (1.2.2)
Requirement already satisfied: matplotlib in c:\users\sudha\appdata\local\programs\p
ython\python310\lib\site-packages (from aif360) (3.10.7)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\sudha\appdata\loca
l\programs\python\python310\lib\site-packages (from pandas>=0.24.0->aif360) (2.9.0.p
ost0)
Requirement already satisfied: pytz>=2020.1 in c:\users\sudha\appdata\local\programs
\python\python310\lib\site-packages (from pandas>=0.24.0->aif360) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in c:\users\sudha\appdata\local\progra
ms\python\python310\lib\site-packages (from pandas>=0.24.0->aif360) (2025.2)
Requirement already satisfied: six>=1.5 in c:\users\sudha\appdata\local\programs\pyt
hon\python310\lib\site-packages (from python-dateutil>=2.8.2->pandas>=0.24.0->aif36
0) (1.17.0)
Requirement already satisfied: joblib>=1.1.1 in c:\users\sudha\appdata\local\program
s\python\python310\lib\site-packages (from scikit-learn>=1.0->aif360) (1.5.2)
Requirement already satisfied: threadpoolctl>=2.0.0 in c:\users\sudha\appdata\local
\programs\python\python310\lib\site-packages (from scikit-learn>=1.0->aif360) (3.6.
0)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\sudha\appdata\local\prog
rams\python\python310\lib\site-packages (from matplotlib->aif360) (1.3.2)
Requirement already satisfied: cycler>=0.10 in c:\users\sudha\appdata\local\programs
\python\python310\lib\site-packages (from matplotlib->aif360) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\sudha\appdata\local\pro
grams\python\python310\lib\site-packages (from matplotlib->aif360) (4.60.1)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\sudha\appdata\local\pro
grams\python\python310\lib\site-packages (from matplotlib->aif360) (1.4.9)
Requirement already satisfied: packaging>=20.0 in c:\users\sudha\appdata\local\progr
ams\python\python310\lib\site-packages (from matplotlib->aif360) (25.0)
Requirement already satisfied: pillow>=8 in c:\users\sudha\appdata\local\programs\py
thon\python310\lib\site-packages (from matplotlib->aif360) (10.4.0)
Requirement already satisfied: pyparsing>=3 in c:\users\sudha\appdata\local\programs
\python\python310\lib\site-packages (from matplotlib->aif360) (3.2.5)
Note: you may need to restart the kernel to use updated packages.
```

Step 2 Load dataset, specifying protected attribute, and split dataset into train and test

In Step 2 we load the initial dataset, setting the protected attribute to be age. We then splits the original dataset into training and testing datasets. Although we will use only the training dataset in this tutorial, a normal workflow would also use a test dataset for assessing the efficacy (accuracy, fairness, etc.) during the development of a machine learning model. Finally, we set two variables (to be used in Step 3) for the privileged (1) and unprivileged (0) values for the age attribute. These are key inputs for detecting and mitigating bias, which will be Step 3 and Step 4.

What is the German Credit Risk dataset?

The original dataset contains 1000 entries with 20 categorial/symbolic attributes prepared by Prof. Hofmann. In this dataset, each entry represents a person who takes a credit by a bank. Each person is classified as good or bad credit risks according to the set of attributes. The link to the original dataset:
https://archive.ics.uci.edu/ml/datasets/Statlog+%28German+Credit+Data%29

Loading dataset

protected_attribute means the attribute on which the bias can occur, basically the attribute you want to test bias for.

privileged_classes means a subset of protected attribute values which are considered privileged from a fairness perspective.

In the german dataset: Old (age >= 25) are the privileged class and Young (age < 25) are the unprivileged class.

Here we have binary membership in a protected group (age) and this is a binary classification problem.

Here, age -> sensitive attribute and Old (age >= 25) is the protected group -> historically systematic advantage group.

1. load the dataset
2. specify the sensitive (protected attribute)
3. print the dataset features
4. split the dataset into training and test

🔧 **Your tasks:**

1. Load the `GermanDataset` from AI Fairness 360.
2. Set the protected attribute (`age`).
3. Define the privileged and unprivileged groups.
4. Print the dataset's features.
5. Split the dataset into training and testing sets (80/20 split).

In [3]:
```
# ⚙️ Step 2 – TODO: Load dataset, define protected attribute, and split into train/

# 👉 TODO 1: Load the German Credit dataset
dataset = GermanDataset()

# 👉 TODO 2: Set the protected attribute (use "age")
protected_attribute = "age"

# 👉 TODO 3: Define privileged and unprivileged groups
privileged_groups = [{protected_attribute: 1}]
unprivileged_groups = [{protected_attribute: 0}]

# 👉 TODO 4: Print dataset features
print("Features: ")
print(dataset.feature_names)

# 👉 TODO 5: Split dataset into training and testing sets (80/20)
train_dataset, test_dataset = dataset.split([0.8], shuffle=True, seed=0)

# Expected Output:
# Features: ['month', 'credit_amount', 'investment_as_income_percentage', 'residenc
# 'number_of_credits', 'people_liable_for', 'sex', 'status=A11', 'status=A12', 'sta
# 'credit_history=A30', 'credit_history=A31', 'credit_history=A32', 'credit_history
# 'purpose=A40', 'purpose=A41', 'purpose=A410', 'purpose=A42', 'purpose=A43', 'purp
# 'purpose=A46', 'purpose=A48', 'purpose=A49', 'savings=A61', 'savings=A62', 'savin
# 'savings=A65', 'employment=A71', 'employment=A72', 'employment=A73', 'employment=
# 'other_debtors=A101', 'other_debtors=A102', 'other_debtors=A103', 'property=A121'
# 'property=A123', 'property=A124', 'installment_plans=A141', 'installment_plans=A1
# 'housing=A151', 'housing=A152', 'housing=A153', 'skill_level=A171', 'skill_level=
# 'skill_level=A174', 'telephone=A191', 'telephone=A192', 'foreign_worker=A201', 'f
```

```
Features:
['month', 'credit_amount', 'investment_as_income_percentage', 'residence_since', 'ag
e', 'number_of_credits', 'people_liable_for', 'sex', 'status=A11', 'status=A12', 'st
atus=A13', 'status=A14', 'credit_history=A30', 'credit_history=A31', 'credit_history
=A32', 'credit_history=A33', 'credit_history=A34', 'purpose=A40', 'purpose=A41', 'pu
rpose=A410', 'purpose=A42', 'purpose=A43', 'purpose=A44', 'purpose=A45', 'purpose=A4
6', 'purpose=A48', 'purpose=A49', 'savings=A61', 'savings=A62', 'savings=A63', 'savi
ngs=A64', 'savings=A65', 'employment=A71', 'employment=A72', 'employment=A73', 'empl
oyment=A74', 'employment=A75', 'other_debtors=A101', 'other_debtors=A102', 'other_de
btors=A103', 'property=A121', 'property=A122', 'property=A123', 'property=A124', 'in
stallment_plans=A141', 'installment_plans=A142', 'installment_plans=A143', 'housing=
A151', 'housing=A152', 'housing=A153', 'skill_level=A171', 'skill_level=A172', 'skil
l_level=A173', 'skill_level=A174', 'telephone=A191', 'telephone=A192', 'foreign_work
er=A201', 'foreign_worker=A202']
```

Step 3 Compute fairness metric on original training dataset

Now that we've identified the protected attribute 'age' and defined privileged and unprivileged values, we can use aif360 to detect bias in the dataset.

1. **Mean Difference**: compare the percentage of favorable results for the privileged and unprivileged groups, subtracting the former percentage from the latter. Use the

BinaryLabelDatasetMetric function and use mean_difference() to print the mean difference

2. **Pearson correlation**: next, measure the correlation between the sensitive attribute and the label (label label_map = {1.0: 'Good Credit', 0.0: 'Bad Credit'})

What to do:

1. Use `BinaryLabelDatasetMetric` to compute the **Mean Difference** between groups.
   - It compares the rate of favorable outcomes (e.g., good credit) between privileged and unprivileged groups.
2. Compute the **Pearson Correlation** between the sensitive attribute (`age`) and the label (`credit outcome`).
   - This measures how strongly `age` influences the label values.

💡 **Interpretation Tips:**

- A large negative or positive mean difference indicates bias.
- A Pearson correlation close to 0 means the label is not strongly correlated with the sensitive attribute.

In [4]:
```
# 📊 Step 3 — TODO: Compute fairness metrics for the original training dataset

# 👉 TODO 1: Create a BinaryLabelDatasetMetric object using the dataset, privileged
metric_orig = BinaryLabelDatasetMetric(train_dataset, privileged_groups=privileged_

# 👉 TODO 2: Compute and print the Mean Difference
mean_diff = metric_orig.mean_difference()
print("Mean Difference:", mean_diff)

# 👉 TODO 3: Compute and print the Pearson Correlation
age = train_dataset.features[:, train_dataset.feature_names.index("age")]
labels = train_dataset.labels.ravel()

pearson_corr = np.corrcoef(age, labels)[0, 1]
print("Pearson Correlation:", pearson_corr)

# Expected Output Example:
# Mean Difference: -0.14945
# Pearson Correlation: -0.12794
```

```
Mean Difference: -0.17972115111019538
Pearson Correlation: -0.15707186416173144
```

Short Response (Conceptual)

- Q1: Based on your Mean Difference result, does the dataset show evidence of bias? Explain briefly (1–2 sentences).
  A1: Yes. The mean difference of –0.1797 indicates that the unprivileged group (age < 25) receives about 17.9% fewer favorable credit outcomes than the privileged group (age

>= 25). Since this value is noticeably far from zero, it provides clear evidence of disparate impact or bias against the younger group.

- Q2: What does the Pearson correlation value tell you about the relationship between age and credit outcome?
  A2: The Pearson correlation of –0.157 indicates a weak negative relationship between age and the credit outcome. This means younger applicants are slightly more likely to receive a negative credit label, but the effect is small. So age is related to the outcome, but not strongly.

Step 4 Mitigate bias by transforming the original dataset

The previous step showed that the privileged group was getting more positive outcomes in the training dataset. Since this is not desirable, we are going to try to mitigate this bias in the training dataset. As stated above, this is called pre-processing mitigation because it happens before the creation of the model.

AI Fairness 360 implements several pre-processing mitigation algorithms.

1. Apply the Reweighing algorithm [1] on the original dataset, which is implemented in the Reweighing class in the aif360 algorithms. This algorithm will transform the dataset to have more equity in positive outcomes on the protected attribute for the privileged and unprivileged groups.
2. Then call the fit and transform methods to perform the transformation, producing a newly transformed training dataset (dataset_transf_train).

[1] F. Kamiran and T. Calders, "Data Preprocessing Techniques for Classification without Discrimination," Knowledge and Information Systems, 2012.

Reweighing: Reweighing is a data preprocessing technique that recommends generating weights for the training examples in each (group, label) combination differently to ensure fairness before classification. The idea is to apply appropriate weights to different tuples in the training dataset to make the training dataset discrimination free with respect to the sensitive attributes. Instead of reweighing, one could also apply techniques (non-discrimination constraints) such as suppression (remove sensitive attributes) or massaging the dataset — modify the labels (change the labels appropriately to remove discrimination from the training data). However, the reweighing technique is more effective than the other two mentioned earlier.

In [5]:
```
# ⚖️ Step 4 – TODO: Mitigate bias using the Reweighing algorithm

# 👉 TODO 1: Create a Reweighing object using privileged and unprivileged groups
from aif360.algorithms.preprocessing import Reweighing
Reweighted = Reweighing(unprivileged_groups=unprivileged_groups, privileged_groups=

# 👉 TODO 2: Fit and transform the training dataset to create a bias-mitigated vers
Reweighted.fit(train_dataset)
```

```
dataset_transf_train = Reweighted.transform(train_dataset)

# # 📄 Expected Output:
# (No printed output — but the variable 'dataset_transf_train' (or your chosen name
# should now contain the reweighted, fair version of the training dataset.)
```

Short Response

- Q3: In your own words, explain what the **Reweighing** algorithm does. Why is it considered a **pre-processing** mitigation technique?
  A3: The Reweighing algorithm computes expected (protected attribute value, label) frequencies under a discrimination-free distribution and assigns instance weights so that the empirical distribution matches this target. By adjusting sample weights rather than changing features or labels, it compensates for the class imbalances in the dataset. It is a pre-processing mitigation method because the reweighting occurs entirely at the data level prior to model training, influencing any classifier trained on it.

Step 5 Compute fairness metric on transformed dataset

Now that we have a transformed dataset, we can check how effective it was in removing bias by using the same metric we used for the original training dataset in Step 3.

1) Repeat Step 3: use the function mean_difference in the BinaryLabelDatasetMetric class and compute the mean difference for the transformed dataset

In [6]:
```
# ✅ Step 5 — TODO: Evaluate fairness after reweighing

# 👉 TODO 1: Use BinaryLabelDatasetMetric again, but this time on the transformed d
metric_transf = BinaryLabelDatasetMetric(dataset_transf_train, privileged_groups=pr

# 👉 TODO 2: Print the mean difference after reweighing
mean_diff_transf = metric_transf.mean_difference()
print("Mean Difference after Reweighing:", mean_diff_transf)


# Expected Output Example:
# Mean Difference after Reweighing: 0.0
# (A result near 0 means the reweighing process successfully reduced bias.)
```

```
Mean Difference after Reweighing: 0.0
```

Reflection

- Q4: Compare the Mean Difference before and after reweighing. What does the change (or lack of change) tell you about the effectiveness of mitigation?
  A4: The Mean Difference before reweighing was around –0.18, indicating substantial bias against the unprivileged group. After reweighing, the Mean Difference moved to 0, meaning that favorable outcomes are now distributed much more evenly across groups.

This shift shows that the bias was successfully mitigated by reweighing the training dataset.